Institución Universitaria

Facultad de Ciencias Exactas y Aplicadas
Física Computacional 1
Robinson Longas Bedoya
robinsonlongas3586@correo.itm.edu.co
robinsonlongas@itm.edu.co

► **Lectura y escritura de archivos**

Bibliografía

- *Computational Physics, Capítulo 2*, Mark Newman.
- *Introducción a la programación con Python. Capítulo 8*, Andrés Marzal, Isabel Gracia y Pedro García.
- *Programming for computations- Python. Sección: 5.5*, Evein Linge and Petter Langtangen.
- *Computational Physics with Python. Capítulo 0*, Eric Ayars.

Entrada y salida de datos: Lectura y escritura de archivos

La interacción con archivos de texto para la lectura de datos de entrada y la exportación de resultados es una tarea fundamental en el cálculo científico y la física computacional.

1. La función `open()` y Modos de Apertura

Para trabajar con un archivo en Python, primero debemos abrirlo mediante la función integrada `open()`, la cual recibe el nombre del archivo y el modo de acceso:

- `'r'` (*Read*): Abre el archivo para lectura (modo por defecto). Genera un error si el archivo no existe.
- `'w'` (*Write*): Abre el archivo para escritura. Sobrescribe completamente el contenido si el archivo ya existe o lo crea si no existe.
- `'a'` (*Append*): Abre el archivo para anexar datos al final sin modificar el contenido preexistente.

2. El Gestor de Contexto (`with`)

En el Python moderno, las operaciones de entrada/salida se gestionan mediante la sentencia `with`. Esto garantiza la liberación adecuada de los recursos del sistema (como los descriptores de archivo) de forma automática al salir del bloque, incluso si ocurren excepciones durante la ejecución:

```
with open('archivo.txt', 'r') as f:
    # Bloque de operacion sobre el archivo
    pass
# El archivo se cierra automaticamente aqui
```

3. Operaciones Básicas de Lectura y Escritura

A continuación se detallan los métodos elementales para la manipulación de información en disco:

- **Lectura línea a línea (`readline()`):** Devuelve una sola cadena de texto correspondiente a la línea actual y avanza el cursor interno.
- **Iteración directa sobre el flujo:** Recorrer directamente el objeto de archivo mediante un bucle `for` permite procesar archivos de gran tamaño de manera altamente eficiente en memoria RAM.
- **Escritura (`write()`):** Escribe una cadena de texto en el archivo. Requiere incluir explícitamente el carácter de salto de línea (`\n`) al final de cada registro.

Ejemplo:

Consideremos como ejemplo un archivo de entrada `masavolumen.txt` que contiene la información de la Tabla 1:

Masa (g)	Volumen (m^3)
1000000	1
500000	0.1
300000	0.2

Tabla 1: Tabla con valores para la masa y el volumen de un cuerpo.

El siguiente programa lee las variables, massa y volumen, calcula la densidad ($\rho = m/V$) y genera un nuevo archivo `resultados.txt`:

in_out.py

```python
"""En este programa se ilustra el uso de la lectura y la escritura de
   ficheros"""

archivo_entrada='../data/masavolumen.txt'
archivo_salida='../data/masavolumendensidad.txt'

#abrimos los archivos
with open(archivo_entrada, 'r') as f_in, open(archivo_salida, 'w') as f_out:
    # 1. Leer y escribir el encabezado
    encabezado = f_in.readline()
    f_out.write(f"{encabezado.strip()}   Densidad (g/m^3)\n")

    # 2. Procesar linea a linea
    for linea in f_in:
        partes = linea.split()
        if partes:  # Omite lineas vacias
            masa = float(partes[0])
            volumen = float(partes[1])
            densidad = masa / volumen
```

```
19
20              # 3. Guardar resultados
21              f_out.write(f"{masa:<10.1f} {volumen:<10.2f} {densidad:<12.2f}\n
    ")
```

Ejemplo: masa inercial vs masa gravitacional

En el contexto de la interacción gravitacional, se tiene que la interacción entre dos cuerpos de masas gravitacionales m_g y M_g está dictaminada por la *ley de gravitación universal*

$$F = -G\frac{m_g M_g}{r^2},$$

donde G es la constante de newton y r la separación entre las masas. Por otro lado, la ley de inercia de Galileo-Newton, implica que $F = m_i a$, donde m_i es la masa inercial que representa la resistencia"de un cuerpo a cambiar su estado de movimiento.

Conceptualmente, la masa inercial y gravitacional son diferentes, sin embargo, el experimento muestra que hay una relación entre ellas, $m_i = m_g$, lo que constituye un principio fundamental en la teoría de la relatividad general, *el principio de equivalencia*.

Para el caso de un péndulo de longitd l que oscila con un periodo T, se puede mostrar que $\frac{m_g}{m_i} = 4{,}02811\frac{l}{T^2}$. En el archjivo `datosL1.txt` se encuentran los datos experimentales medidos pra l y T. En el sigueinte programa, se leen esos datos y se calcula la razpkn entra la masa inercial y gravitacional.

masaIvsmasaG.py

```
1  """En este programa se ilustra el uso de la lectura de datos para
       graficarlos"""
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  datos= np.loadtxt('../data/datosL1.txt')
6  l = datos[:,0] # la primera columna es la longitud
7  t = datos[:,1] # la segunda columna es el tiempo
8  #calculo de la masa gravitacional/ masa inercial
9  mgmi = 4.0281 * l / (t**2)
10
11 # plot
12 plt.figure(figsize=(8, 6))
13 plt.plot(t, mgmi, color='red', ls='-', lw=3, label=r'$\boldsymbol{Datos}$')
14 plt.yscale('log')
15 plt.ylim(0.1,10)
16 plt.axhline(1, color='black', ls='--', lw=3, label=r'$\boldsymbol{Esperado}$
       ')
17 plt.xlabel("tiempo(s)", size=20)
18 plt.ylabel(r'$\boldsymbol{m_g/m_i}$', size=20)
19 plt.legend(prop={'size': 15}, loc='upper left', frameon=True)
20 plt.savefig("mgvsmi.pdf")
21 plt.show()
```

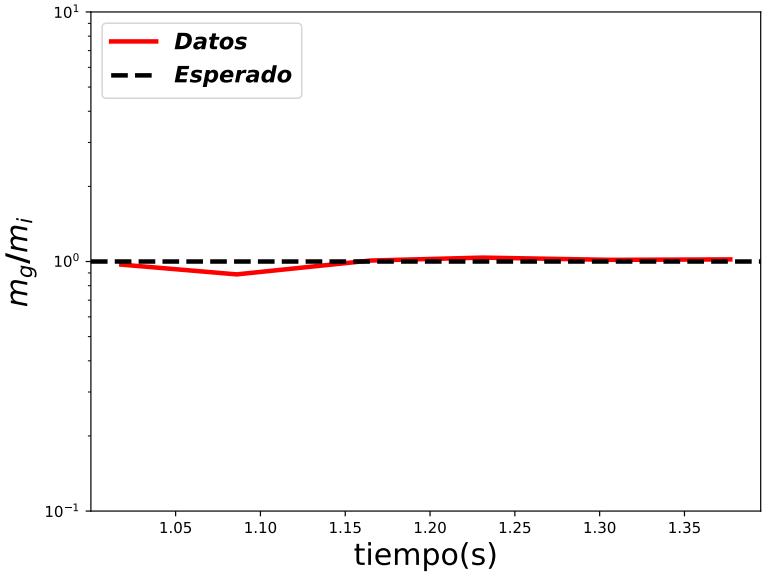


Figura 1: Prueba experimental del principio de equivalencia.

Ejemplo:

Para finalizar, consideremos un archivo de datos experimental denominado `microphones.txt`, cuyo contenido está organizado por columnas separadas por tabulaciones o espacios:

#Frequency	Mic 1	Mic 2
10.000	0.654	0.192
11.000	0.127	0.032
12.000	0.120	0.030
13.000	0.146	0.031
14.000	0.155	0.033
15.000	0.175	0.036
⋮	⋮	⋮

Tabla 2: Estructura del archivo `datosmicrofono.txt`.

`microfono.py`

```
import numpy as np
import matplotlib.pyplot as plt

# Cargar las columnas directamente en variables independientes
frequency, mic1, mic2 = np.loadtxt('../data/datosmicrofono.txt', unpack=True
    )

# graficar
plt.figure(figsize=(8, 5))
```

```
plt.plot(frequency, mic1, 'r-', label='Micrófono 1')
plt.plot(frequency, mic2, 'b-', label='Micrófono 2')

plt.xlabel('Frecuencia (Hz)',size=15)
plt.ylabel('Amplitud (cm)',size=15)
plt.legend(loc='upper right')
plt.grid(True, ls=':', alpha=0.6)
plt.savefig("../Clases/Introduccion_Python/microfono.pdf")
plt.show()
```

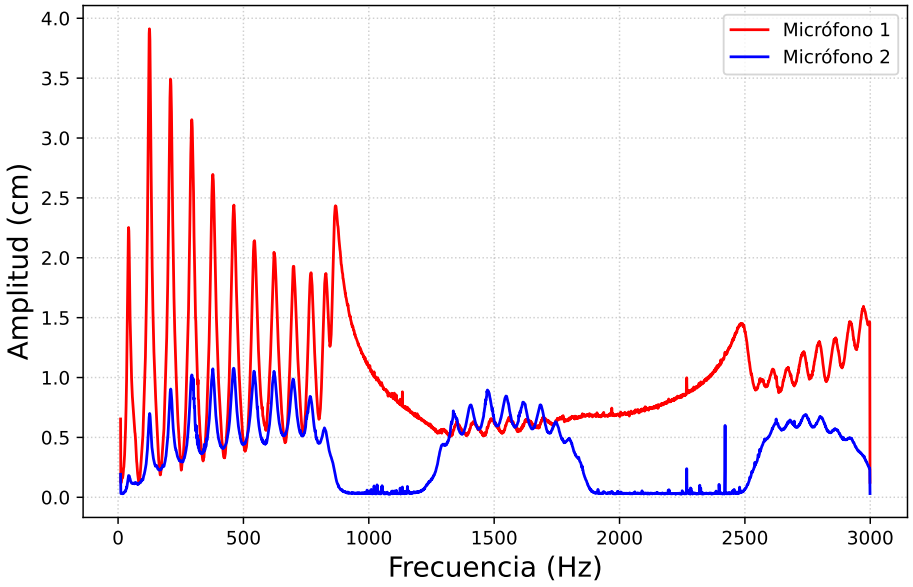


Figura 2: Datos de los micrófonos.

- **`unpack=True`** le indica a la función que el archivo contiene datos dispuestos en columnas y que debe devolver cada una de ellas como un arreglo independiente de NumPy.
- **`np.loadtxt()`** ignora cualquier línea que comience con el carácter `#`, asumiendo que se trata de un encabezado o comentario.
- Se requiere que todas las filas tengan exactamente el mismo número de columnas y que todos los elementos sean valores numéricos.

Problemas:

1. La forma más útil de analizar los datos mostrados en la Figura 2 es calculando la razón entre las señales de ambos micrófonos. Grafique esta razón (Mic 1/Mic 2) en función de la frecuencia en el eje x. Asegúrese de dar un acabado adecuado a la gráfica ajustando las escalas, las etiquetas de los ejes y el título correspondiente.

2. El archivo `radiative.txt` contiene dos columnas de datos: la primera corresponde a los conteos registrados por un contador Geiger y la segunda al tiempo en segundos.

(a) Genere una gráfica adecuada y clara de estos datos.

(b) Si los datos siguen una curva exponencial, al graficar el logaritmo natural de los conteos (o al representar los datos originales en una escala semilogarítmica) se debería obtener una línea recta. Determine si este es el caso y explique su conclusión respaldándola con una gráfica apropiada.

3. Los datos en el archivo `radiative.txt` corresponden a mediciones reales de un experimento de decaimiento radiactivo; la primera columna es el número de decaimientos N y la segunda es el tiempo t en segundos. Nos interesa determinar el tiempo de vida media $t_{1/2}$ del ^{137}Ba.

   El decaimiento sigue la ecuación teórica:

$$N(t) = N_0 e^{-\lambda t} \quad \text{donde} \quad \lambda = \frac{\ln 2}{t_{1/2}}$$

   Utilizando las técnicas aprendidas en este capítulo, cargue los datos del archivo `radiative.txt` en variables adecuadamente nombradas en una sesión de Python. Experimente con diferentes valores de N_0 y λ (o $t_{1/2}$) y grafique la curva teórica sobre los datos experimentales. Intente ajustar los valores manualmente hasta que la curva coincida adecuadamente con los datos. ¿Cuál es su mejor estimación para $t_{1/2}$?